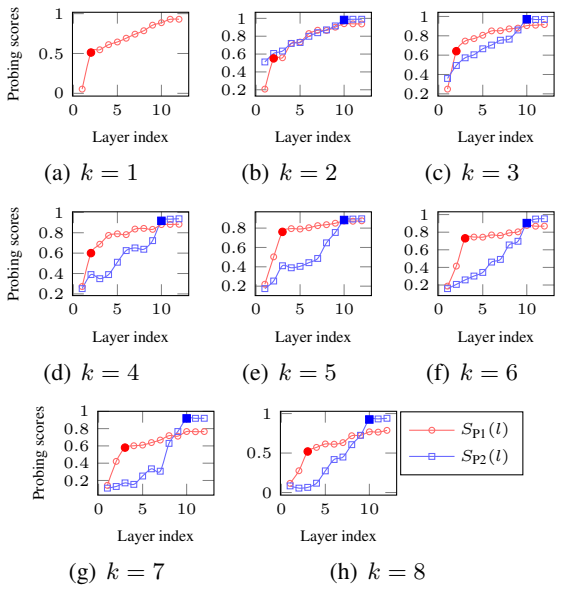
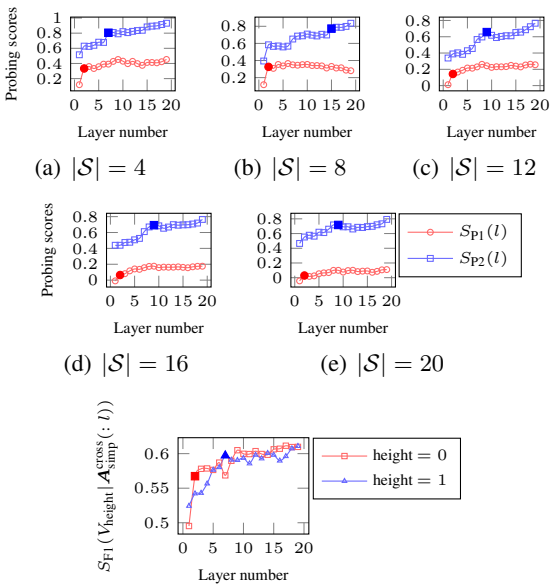


Figure 6: Two probing scores on partial attentions across 12 layers. The x-axis represents the layer index of GPT-2$_{\text{FT}}$, and the y-axis represents the probing scores. Similarly, we test different k from 1 to 8 ($m = 16$ by default). Results show that each layer of GPT-2$_{\text{FT}}$ focuses on different steps of the synthetic reasoning task.

Figure 6 shows the layer-wise probing scores for each k for GPT-2$_{\text{FT}}$ models. Observing S_{P1}, i.e., selecting top-k numbers, we notice that GPT-2$_{\text{FT}}$ quickly achieves high scores in initial layers and then, S_{P1} increases gradually. Observing S_{P2}, i.e., selecting the k-th smallest number from top-k numbers, we notice that GPT-2$_{\text{FT}}$ does not achieve high scores until layer 10. This reveals how GPT-2$_{\text{FT}}$ solves the task internally. The bottom layers find out the top-k numbers, and the top layers select the k-th smallest number among them. Results in Figure 5 also support the above findings.

LLaMA on ProofWriter and ARC ($\boldsymbol{A}^{\text{cross}}_{\text{simp}}$). Similarly, we report layer-wise probing scores $S_{\text{P1}}(l)$ and $S_{\text{P2}}(l)$ for LLaMA under the 4-shot setting. We further report $S_{\text{F1}}(V_{\text{height}}|\boldsymbol{A}^{\text{cross}}_{\text{simp}}(:\, l))$ for nodes with height $= 0$ and height $= 1$ to show if the statement at height 0 is processed in LLaMA before the statement at height 1.

The layer-wise probing results for ProofWriter are shown in Figure 7(a-e). We find that similar to GPT-2, probing results for 4-shot LLaMA reach a plateau at an early layer (layer 2 for $S_{\text{P1}}(l)$), and at middle layers for $S_{\text{P2}}(l)$. This observation holds as we vary$|\mathcal{S}|$ from 4 to 20. This shows that similar to GPT-2, LLaMA first tries to identify useful statements in the bottom layers.



(f) Height probing

Figure 7: Layer-wise probing results on ProofWriter. x-axis represents the number of layers used for probing. We can find that $S_{\text{P1}}(l)$ scores reach the plateau quickly and $S_{\text{P2}}(l)$ scores increase smoothly till the middle layers. This indicates that useful statement selection is mainly finished in the bottom layers and the reasoning step is decided in the middle layers. Results of $S_{\text{F1}}(V_{\text{height}}|\boldsymbol{A}^{\text{cross}}_{\text{simp}}(:\, l))$ shows that statements at height 0 are identified by LLaMA in bottom layers, and statement at height 1 are identified later in middle layers.

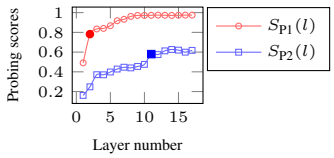


Figure 8: Layer-wise probing results on ARC (depth=1). Similar to that of ProofWriter, we can find that $S_{\text{P1}}(l)$ scores reach the plateau quickly and $S_{\text{P2}}(l)$ scores increase smoothly till the middle layers.

Then, it focuses on predicting the reasoning steps given the useful statements. Figure 7(f) shows how $S_{\text{F1}}(V_{\text{height}}|\boldsymbol{A}^{\text{cross}}_{\text{simp}}(:\, l))$ varies across layers. LLaMA identifies the useful statements from all the input statements (height 0) immediately in layer 2. Then, LLaMA gradually focuses on these statements and builds the next layer of the reasoning tree (height 1) in the middle layers.

The layer-wise probing results for ARC are in Figure 8. Similar to ProofWriter, the $S_{\text{P1}}(l)$ scores on ARC for 4-shot LLaMA reach a plateau at an early layer (layer 2), and at middle layers for $S_{\text{P2}}(l)$. This also shows that LLaMA tries to identify useful statements in the bottom layers and focuses on the next reasoning steps in the higher layers.

5 Do LMs Reason Using $\boldsymbol{A}_{\text{simp}}$?

Our analysis so far shows that LMs encode the reasoning trees in their attentions. However, as argued by Ravichander et al. (2021); Lasri et al. (2022); Elazar et al. (2021), this information might be accidentally encoded but not actually used by the LM for inference. Thus, we design a causal analysis for GPT-2 on the k-th smallest element task to show that LMs indeed perform reasoning following the reasoning tree. The key idea is to prove that the attention heads that contribute to $\boldsymbol{A}_{\text{simp}}$ are useful to solve the reasoning task, while those heads that are irrelevant (i.e., independent) to $\boldsymbol{A}_{\text{simp}}$ are not useful in the reasoning task.

Intuitively, for the k-th smallest element task, attention heads that are sensitive to the number size (rank) are useful, while heads that are sensitive to the input position are not useful. Therefore, for each head, we calculate the attention distribution on the test data to see if the head specially focuses on numbers with a particular size or position. We use the entropy of the attention distribution to measure this[9]: small entropy means that the head focuses particularly on some numbers. We call the entropy with respect to number size as *size entropy* and that with respect to input position as *position entropy*. The entropy of all the heads in terms of number size and position can be found in Appendix B.5.

We prune different kinds of attention heads in order of their corresponding entropy values and report the test accuracy on the pruned GPT-2.[10] Results with different k are shown in Figure 9. We find that the head with a small size entropy is essential for solving the reasoning task. Dropping 10% of this kind of head, leads to a significant drop in performance on the reasoning task. The heads with small position entropy are highly redundant. Dropping 40% of the heads with small position entropy does not affect the test accuracy much. Especially when $k = 1$, dropping 90% position heads could still promise a high test accuracy.

These results show that heads with small size entropy are fairly important for GPT-2 to find k-th smallest number while those with small position entropy are useless for solving the task. Note that the reasoning tree G is defined on the input number size and it is independent of the number position. `MechanisticProbe` detects the information of G from attentions. Thus, our probing scores would be affected by the heads with small size entropy but would not be affected by heads with small position entropy. Then, we can say that changing our probing scores (via pruning heads in terms of size entropy) would cause the test accuracy change. Therefore, we say that there is a causal relationship between our probing scores and LM performance, and LMs perform reasoning following the reasoning tree in $\boldsymbol{A}_{\text{simp}}$.

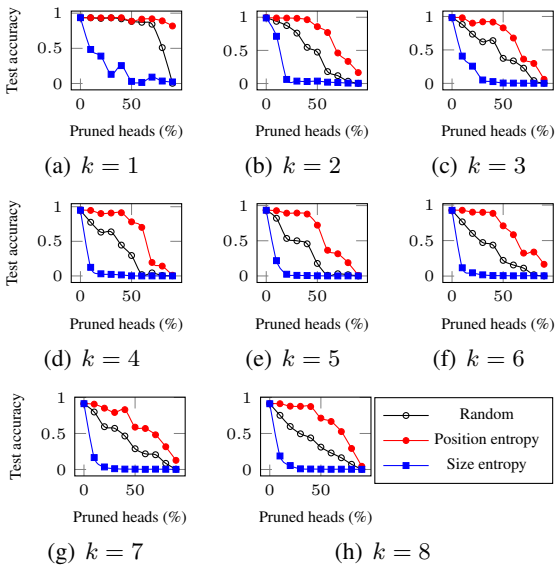


Figure 9: Test accuracy on the reasoning task for different pruning rates. The x-axis represents pruning rates: ranging from 0% to 90%, and the y-axis represents the test accuracy. We prune heads in the ascending order of their size/position entropy. Results show that heads with small size entropy are essential to the test accuracy while those with small position entropy are useless.

6 Correlating Probe Scores with Model Accuracy and Robustness

Our results show that LMs indeed reason mechanistically. But, is mechanistic reasoning necessary for LM performance or robustness? We attempt to answer this question by associating the probing scores with the performance and robustness of LMs. Given that finetuned GPT-2 has a very high test accuracy on the synthetic task and LLaMA does not perform as well on ARC, we conduct our analysis mainly with LLaMA on the ProofWriter task.

Accuracy. We randomly sample 64 to 128 examples from the dataset and test 4-shot LLaMA on these examples. We calculate their test accuracy

[9]We regard the attentions $\mathbb{E}[\boldsymbol{A}_{\text{simp}}(l,h)]$ as probabilities, and calculate the entropy of head h at layer l using them.

[10]We gradually prune heads, and the order of pruning heads is based on the size/position entropy. For example, for the *size entropy* pruning with 50% pruned heads, we remove attention heads that have the top 50% smallest size entropy.

Table 3: Pearson correlation coefficient between our probing scores and test accuracy for 4-shot LLaMA.

Pearson correlation coefficient ρ ($\times 100\%$)	
ρ (S_{P1}, S_{P2})	0.01
ρ (Test accuracy, S_{P1})	27.42
ρ (Test accuracy, S_{P2})	71.13

and the two probing scores S_{P1} and S_{P2}. We repeat this experiment 2048 times. Then, we calculate the correlation between the probing scores and test accuracy. From Table 3, we find that test accuracy is closely correlated with S_{P2}. This implies that when we can successfully detect reasoning steps of useful statements from LM's attentions, the model is more likely to produce a correct prediction.

Robustness. Following the same setting, we also associate the probing scores with LM robustness. In order to quantify model robustness, we randomly corrupt one useless input statement for each example, such that the prediction would remain unchanged.[11] We measure robustness by the decrease in test accuracy after the corruption.

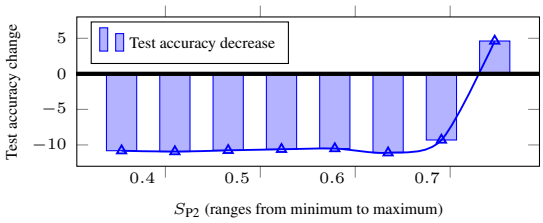


Figure 10: Histogram for LM prediction robustness and S_{P2}. We measure robustness by the change in accuracy on the corrupted dataset. We present results with 8 bins. Results show that LLaMA is more robust to noise on examples with higher probing scores S_{P2}.

Figure 10 shows that if S_{P2} is small (less than 0.7), the prediction of LLaMA could be easily influenced by the noise (test accuracy decreases around 10%). However, if the probing S_{P2} is high, LLaMA is more robust, i.e., more confident in its correct prediction (test accuracy increases around 4%). This provides evidence that if the LM encodes the gold reasoning trees, its predictions are more reliable (i.e., robust to noise in the input).

7 Related Work

Attention-based analysis of LMs. Attention has been popularly used to interpret LMs (Vig and Belinkov, 2019; DeRose et al., 2021; Bibal et al., 2022). A direct way for interpreting an attention-based model is to visualize attentions (Samaran et al., 2021; Chefer et al., 2021). But the irrelevant, redundant, and noisy information captured by attentions makes it hard to find meaningful patterns. Alternatively, accumulation attentions that quantify how information flows across tokens can be used for interpretation (Abnar and Zuidema, 2020; Eberle et al., 2022). However, for casual LMs, information flows in one direction, and it causes an over-smoothing problem when the model is deep.[12] To tackle this, other works propose new metrics and analyze attentions using them (Ethayarajh and Jurafsky, 2021; Liu et al., 2022). However, these metrics are proposed under specific scenarios, and these are not useful for detecting the reasoning process in LMs. We address this challenge by designing a more structured probe that predicts the reasoning tree in LMs.

Mechanistic Interpretability. Mechanistic interpretation explains how LMs work by reverse engineering, i.e., reconstructing LMs with different components (Räuker et al., 2022). A recent line of work provides interpretation focusing on the LM's weights and intermediate representations (Olah et al., 2017, 2018, 2020). Another line of work interprets LMs focusing on how the information is processed inside LMs (Olsson et al., 2022; Nanda et al., 2023). Inspired by them, Qiu et al. (2023) attempts to interpret how LMs perform reasoning. However, existing explorations do not cover the research problem we discussed.

8 Conclusion

In this work, we raised the question of whether LMs solve procedural reasoning tasks step-by-step within their architecture. In order to answer this question, we designed a new probe that detects the oracle reasoning tree encoded in the LM architecture. We used the probe to analyze GPT-2 on a synthetic reasoning task and the LLaMA model on two natural language reasoning tasks. Our empirical results show that we can often detect the information in the reasoning tree from the LM's attention patterns, lending support to the claim that LMs may indeed be reasoning "mechanistically".

[11] We do it by adding negation: corrupting S_i as ["That" $\circ$ S_i $\circ$ " is false"], where $\circ$ means string concatenation.

[12] The issue is that all token representations are dominated by the first token. Detailed discussion is in Appendix A.